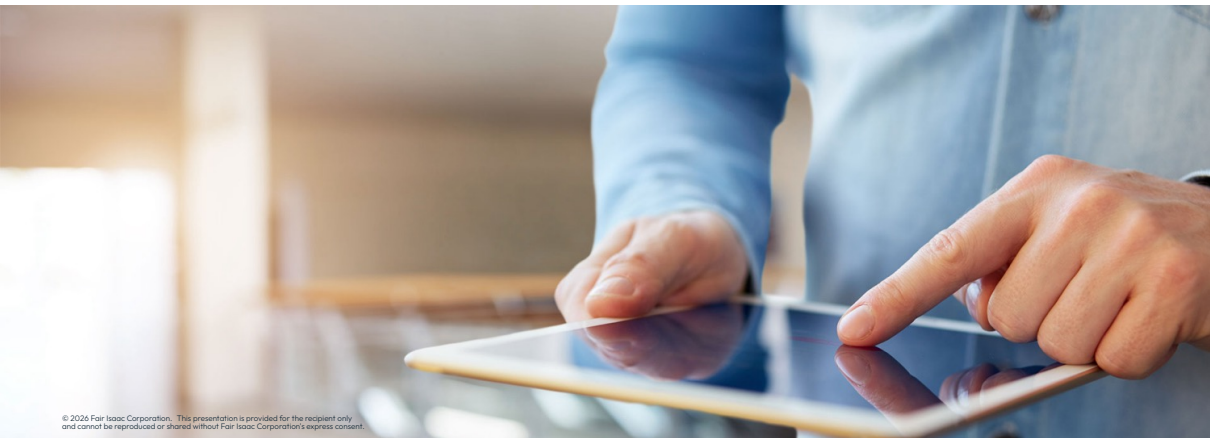


Using Numpy





Using the numerical library
NumPy

Introducing *Xpress* arrays

- The *NumPy* library allows for creating and using arrays of any order and size for **efficiency and compactness** purposes:
 - *NumPy* operations are implemented in lower level C, which allows for much more efficient loop operations when creating expressions (linear and nonlinear) with variables, and constraints
 - `xpress.ndarray` is a subclass of `numpy.ndarray` which customizes the `<`, `>=` and `==` operators
 - **In all other ways, `xpress.ndarray` behaves the same as `numpy.ndarray`**

Introducing *Xpress* arrays

- The *NumPy* library allows for creating and using arrays of any order and size for **efficiency and compactness** purposes:
 - *NumPy* operations are implemented in lower level C, which allows for much more efficient loop operations when creating expressions (linear and nonlinear) with variables, and constraints
 - `xpress.ndarray` is a subclass of `numpy.ndarray` which customizes the `<`, `>=` and `==` operators
 - **In all other ways, `xpress.ndarray` behaves the same as `numpy.ndarray`**
- The `problem.addVariables()` function in its simplest usage directly returns *Xpress* array of variables with one or more indices:

- The *Xpress* array declarations:

```
x = xp.array([p.addVariable(name='v({0})'.format(i)) for i in range(20)]).reshape(5, 4)
y = xp.array([p.addVariable(lb=-1, ub=1) for i in range(1000)])
```

- ...can be written equivalently in the compact form by passing integer arguments to `p.addVariables()`:

```
x = p.addVariables(5, 4, name='v')
y = p.addVariables(1000, lb=-1, ub=1)
```

Using *Xpress* arrays

- *NumPy* operations can be replicated on each element of an array, leveraging its *vectorization* and *broadcasting* features:
 - These operations can be carried out on arrays of any number of dimensions, and can be aggregated at any level
 - To *broadcast* the right-hand side 1 to all elements of the array, creating the set of constraints:
$$x[i] + y[i] \leq 1, \forall i \in S:$$

`constr2 = x + y <= 1`

Using *Xpress* arrays

- *NumPy* operations can be replicated on each element of an array, leveraging its *vectorization* and *broadcasting* features:
 - These operations can be carried out on arrays of any number of dimensions, and can be aggregated at any level
 - To *broadcast* the right-hand side 1 to all elements of the array, creating the set of constraints:
 $x[i] + y[i] \leq 1, \forall i \in S$
`constr2 = x + y <= 1`
 - Creating two three-dimensional arrays of variables involved in a set of constraints $z_{ijk}^2 \leq 1 + t_{ijk}, \forall i, j, k$:
`z = p.addVariables(4, 5, 10)`
`t = p.addVariables(4, 5, 10, vartype=xp.binary)`
`p.addConstraint(z**2 <= 1 + t)`

Products of *Xpress* arrays

- The `xpress.Dot()` operator is useful for carrying out aggregate operations on vectors and matrices in arrays containing Xpress variables and expressions:
 - When handling variables or expressions, use the `xp.Dot()` operator rather than *NumPy*'s *dot* operator
 - Examples where z is one-dimensional:
 - Restrict squared norm: $\sum_i z_i^2 \leq 1$
 - Bound quadratic expression: $\sum_{i,j} (t_i - z_i) Q_{ij} (t_j - z_j) \leq 1$

```
p.addConstraint(xp.Dot(z, z) <= 1)
Q = np.random.random(20, 20)
p.addConstraint(xp.Dot((t-z), Q, (t-z)) <= 1)
```

Products of *Xpress* arrays

- The `xpress.Dot()` operator is useful for carrying out aggregate operations on vectors and matrices in arrays containing Xpress variables and expressions:
 - For multi-dimensional arrays, the size of the last dimension of the first array must match the size of the penultimate dimension of the second vector:
 - Matrix multiplication creating constraints $\sum_k a_{ik} b_{kj} \leq 10, \forall i, j$:

```
a = p.addVariables(4,6, name="a")
b = p.addVariables(6,2, name="b")
p.addConstraint(xp.Dot(a,b) <= 10)
```
 - Yields a 4x2 matrix creating 8 new constraints
 - Rules are the same as for the *NumPy* dot operator, except that there is no limit on the number of arguments



Thank You

www.fico.com/optimization